Machine Coding: Autocomplete Search Bar

- **Problem:** Build a product search box with an autocomplete dropdown. While typing, it shows up to 8 suggestions from an API (`dummyjson.com/products/search`), debounced, with the matching text highlighted. When the input is empty, it shows the last 5 recent searches (saved in localStorage, removable). The dropdown works fully with the keyboard (↑ ↓ Enter Escape Delete) and screen readers (WAI-ARIA combobox). Choosing a suggestion or pressing Enter shows product results below; default products load on first visit.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Sources, limits, keyboard rules decided
Use cases + architecture + state	6 min	Combobox state: query, isOpen, activeIndex
Input + debounced suggestions + click to select	12 min	Working autocomplete
Keyboard navigation + ARIA combobox	12 min	Accessible dropdown
Recent searches (localStorage), highlight, results list	10 min	Full feature
Abort/cache, loading/error/empty, blur handling	7 min	Robust
Keyboard-only test + wrap-up	8 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"Focus never leaves the input. The highlighted option is just an index in state, exposed with* `aria-activedescendant` ,

so typing keeps working and screen readers still announce the highlighted option." Then demo with the keyboard only.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Type ≥ 2 characters → suggestions from the API after a 300 ms pause, max 8, matching part bold.
- Empty input + focus → recent searches (max 5, newest first, no duplicates), each removable.
- Select by click, or by ↑/↓ + Enter; Enter with nothing highlighted searches the typed text.
- Escape closes the dropdown; a second Escape clears the input. Tab / clicking outside closes it.
- Selected/submitted term → saved to recent + product results shown below; default products on load.
- States in the dropdown: "Searching...", "No suggestions for 'x'", "Couldn't load suggestions."

Non-functional requirements

Concern	What it means here
Performance	Debounce (300 ms), abort stale requests, cache by term, small `select=` payload for suggestions.
Scalability	Suggestion source is one function (`fetchSuggestions`); the combobox doesn't care where options come from.
Error handling	`response.ok` + shape checks; suggestion errors shown in the dropdown, results errors with Retry; corrupted recent history ignored.
Accessibility	`role="combobox"` + `aria-expanded` / `aria-controls` / `aria-activedescendant` / `aria-autocomplete="list"` ; `role="listbox"` + `role="option"` + `aria-selected` ; live region "8 suggestions available"; label for the input.
Reusability	`useDebounce` , `useSuggestions` , `useRecentSearches` , pure `addRecent` / `splitMatch` .
Extensibility	Grouped results (products / categories), server-side "popular searches", infinite suggestions.

Clarifying questions to ask the interviewer

1. Where do suggestions come from: a local list or an API? How many should we show?
2. Minimum characters and debounce delay?
3. Should recent searches be stored? Where, how many, and can users delete them?
4. What happens on Enter with nothing highlighted: search the text, or select the first suggestion?
5. Should the matching text be highlighted? Prefix match or "contains"?
6. Is keyboard / screen-reader support required (the ARIA combobox pattern)?

Assumptions we will build with

- API suggestions: ≥ 2 chars, 300 ms debounce, max 8 (title + category).
- Recent searches: max 5 in localStorage, deduped case-insensitively, removable (× or Delete key).
- Enter with no highlight = search the typed text; ↑/↓ wrap around.
- Highlight the first case-insensitive "contains" match.
- Follow the WAI-ARIA 1.2 combobox pattern (list autocomplete, focus stays in the input).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Focus the empty input (with history)	Dropdown "Recent searches" with up to 5 items
Focus the empty input (no history)	Nothing opens
Type "p"	Nothing (below 2 characters)
Type "pho"	"Searching..." → 8 suggestions with "pho" bold; live region "8 suggestions available"
ArrowDown / ArrowUp	Moves the highlight (wraps at the ends); focus stays in the input; `aria-activedescendant` updates
Mouse hover over an option	Highlights it (same state as the keyboard)
Enter with an option highlighted	Selects it: input = title, dropdown closes, results load, saved to recent
Enter with nothing highlighted	Searches the typed text
Click an option	Same as Enter on it (no blur-before-click problem)
Escape (open) / Escape (closed)	Closes / clears the input
Tab or click outside	Closes
Delete on a highlighted recent item, or click ×	Removes it from history (and storage)
Type "xyzq"	`No suggestions for "xyzq"`
API fails	"Couldn't load suggestions." in the dropdown; typing again retries
Type fast "phone"	One request after the pause; older requests aborted
Search the same term again	Suggestions from the cache
Reload	Recent searches still there; default products load

3. Design the architecture

Module	Responsibility
`App`	Owns the submitted query + recent searches; wires the combobox to the results
`components/Autocomplete.jsx`	The combobox: input, dropdown, keyboard + mouse handling, ARIA
`components/ProductResults.jsx`	Fetches and shows product cards for the submitted query (loading/error/empty/Retry)
`hooks/useDebounce.js`	Debounced value
`hooks/useSuggestions.js`	Suggestions for a term: abort stale, cache, status
`hooks/useRecentSearches.js`	Recent list state + localStorage persistence
`utils/search.js`	Pure `addRecent`, `parseRecent`, `splitMatch`

Module	Responsibility
`api/products.js`	`fetchSuggestions` (small payload), `fetchProducts` (cards)

```
App  (submitted, recent via useRecentSearches)
├── <Autocomplete recent onRemoveRecent onSearch>
│     query, isOpen, activeIndex (state)
│     debounced = useDebounce(query) → useSuggestions(term) → api.fetchSuggestions
│     options = query empty ? recent : suggestions      (derived)
│     <input role="combobox" aria-activedescendant> + <ul role="listbox"> <li role="option">
└── <ProductResults query={submitted}> → api.fetchProducts
```

Start simple. First version: input + `.filter()` on a local array + a clickable list. Then the API + debounce, then the keyboard + ARIA, then recent searches and the polish.

4. Design the state / data model

State	Where	Purpose
`query`	Autocomplete	What's in the input (every keystroke)
`isOpen`	Autocomplete	Whether the user wants the dropdown open (focus/typing vs Escape/blur)
`activeIndex`	Autocomplete	Highlighted option (−1 = none)
`status`, `suggestions`, `error`	useSuggestions	Suggestion request state for the debounced term
`cache` (ref)	useSuggestions	`Map<term, suggestions>`
`recent`	useRecentSearches (App)	Recent terms, persisted
`submitted`	App	The query the results show

```
const [query, setQuery] = useState("");
const [isOpen, setIsOpen] = useState(false);
const [activeIndex, setActiveIndex] = useState(-1);

// derived
const typed = query.trim();
const mode = typed === "" ? "recent" : typed.length >= MIN_LENGTH ? "suggest" : "none";
const options = mode === "recent" ? recent.map(toRecentOption) : mode === "suggest" ?
suggestions.map(toOption) : [];
const active = activeIndex < options.length ? activeIndex : -1;  // never points past the list
const showPopup = isOpen && (options.length > 0 || message !== "");
```

Senior note

- **Derived, not stored:** the first version stored `suggestion` in state and filtered it in an effect, which is one render behind and can disagree with `apiData`. Here `options`, `mode`, `showPopup`, the safe `active` index and the message are all computed during render.
- **`isOpen` is user intent, `showPopup` is the result:** the user may "want it open" but there is nothing to show; deriving `showPopup` avoids an empty box.
- **Ownership:** the combobox owns typing and highlight state; `App` owns what was searched and the history (shared with results). The recent list is passed down; the combobox reports removals via a callback.

5. Design the data structures and contracts

Suggestion (small payload via `select=`):

```
{ "id": 121, "title": "iPhone 5s", "category": "smartphones" }
```

Option (what the listbox renders): `{ key: "product-121", label: "iPhone 5s", meta: "smartphones" }` or `{ key: "recent-laptop", label: "laptop", recent: true }`. Keys are prefixed so a recent term and a product can never collide.

Recent searches in storage: `localStorage["recentSearches"] = ["laptop", "Family Tree Photo Frame"]` (validated on read).

Contracts

```
fetchSuggestions(query, { signal }) → Promise<{ id, title, category }[]>   // GET ...?
q=&limit=8&select=title,category
fetchProducts(query, { signal })    → Promise<{ id, title, description, price, thumbnail }[]>
useSuggestions(term) → { status: "idle"|"loading"|"success"|"error", suggestions, error }
useRecentSearches()  → { recent, add(term), remove(term) }
addRecent(list, term) → new list (trimmed, deduped case-insensitively, max 5)
splitMatch(text, query) → [{ text, match }]
<Autocomplete recent onRemoveRecent={(term)=>…} onSearch={(term)=>…} />
```

Option ids: `${useId()}-option-${i}`, which are unique even with two comboboxes on a page and are what `aria-activedescendant` points to.

6. Trace the data flow

User types "pho", presses ↓ ↓ Enter:

```
Keystrokes p, h, o → setQuery, setIsOpen(true), setActiveIndex(-1)
  ↓ mode "suggest" (3 chars); typed ≠ debounced term → message "Searching..."
  ↓ 300 ms later: debounced "pho" → useSuggestions: cache miss → abort old → fetch ?q=pho&limit=8
  ↓ response → cache.set("pho") → status success → options = 8 suggestions, "Pho" in <mark>
  ↓ live region: "8 suggestions available"
↓ ArrowDown ×2 → activeIndex 1 → aria-activedescendant="…-option-1", aria-selected=true, scrolled into
view
```

```
Enter → select("Family Tree Photo Frame")
  ↓ setQuery(title), close, onSearch(title)
  ↓ App: setSubmitted(title), addRecent → localStorage updated
  ↓ ProductResults: abort old → fetchProducts(title) → cards render
```

7. Important technical challenges

Challenge 1: keyboard navigation with focus staying in the input

- *Problem:* Moving real focus to the options (`tabIndex` + `.focus()`) stops typing: the user presses ↓, then types "e", and the letter goes nowhere. Without arrow-key support, keyboard users can't pick a suggestion at all.
- *Solution:* Keep focus in the input; track the highlighted index in state; expose it with `aria-activedescendant` .

```
case "ArrowDown":
  e.preventDefault();                                  // don't move the text cursor
  setIsOpen(true);
  if (options.length) setActiveIndex(active + 1 >= options.length ? 0 : active + 1);
  break;
case "Enter":
  e.preventDefault();
  if (showPopup && active >= 0) select(options[active].label);
  else if (typed) select(typed);
  break;
<input role="combobox" aria-expanded={showPopup} aria-controls={listboxId}
       aria-activedescendant={showPopup && active >= 0 ? optionId(active) : undefined} />
```

Challenge 2: blur vs click (the dropdown closes before the click lands)

- *Problem:* Clicking an option first blurs the input (`mousedown` → blur → our `onBlur` closes the dropdown), so the option disappears and the `click` never fires. The first version worked around it for recent searches by selecting in `onMouseDown` , which also fires on a press the user drags away from.
- *Solution:* Prevent the input from losing focus on `mousedown` inside the dropdown, and select on `click` .

```
<li role="option"
    onMouseDown={(e) => e.preventDefault()}   // input keeps focus → no blur → dropdown stays
    onMouseEnter={() => setActiveIndex(i)}
    onClick={() => select(option.label)}>
```

Challenge 3: async suggestions done right

- *Problem:* `useCallback(debounce(fetchApiData, 600), [])` needs an eslint-disable and freezes the first render's closure; fast typing can show results for an older query; `fetch` doesn't throw on 404; and `suggestion.length > 1` hides the dropdown when there is exactly **one** match.

- *Solution:* A debounced *value*, a hook that aborts stale requests and caches, `ok`/shape checks, and `options.length > 0`.

```
const debounced = useDebounce(typed, 300);
const term = debounced.length >= MIN_LENGTH ? debounced : "";
const { status, suggestions } = useSuggestions(term);       // AbortController + Map cache inside
const showPopup = isOpen && (options.length > 0 || message !== "");
```

Challenge 4: recent searches and highlighting safely

- *Problem:* `JSON.parse(localStorage...)` crashes on corrupted data; duplicates with different case pile up ("Phone", "phone"); highlighting with `new RegExp(query)` throws on "c++" or "(", and `dangerouslySetInnerHTML` for `<b>` tags opens XSS.
- *Solution:* Pure helpers: validate on read, dedupe case-insensitively, and split the string with `indexOf` (no regex, no HTML injection).

```
export function addRecent(list, term) {
  const value = term.trim();
  if (!value) return list;
  return [value, ...list.filter((t) => t.toLowerCase() !== value.toLowerCase())].slice(0, MAX_RECENT);
}
export function splitMatch(text, query) {
  const q = query.trim();
  const index = q ? text.toLowerCase().indexOf(q.toLowerCase()) : -1;
  if (index === -1) return [{ text, match: false }];
  return [{ text: text.slice(0, index), match: false }, { text: text.slice(index, index + q.length),
match: true },
          { text: text.slice(index + q.length), match: false }].filter((p) => p.text);
}
```

8. Implement in the correct order

#	Step	Working result
1	Controlled input + local array + `.filter()` + clickable list	Basic autocomplete
2	API `fetchSuggestions` + `useDebounce` + min length	Real suggestions
3	`useSuggestions`: status, abort, cache, `ok` check	Robust fetching
4	`isOpen` + derived `showPopup`; close on blur; mousedown `preventDefault`	Correct open/close
5	`activeIndex` + ArrowUp/Down/Enter/Escape/Tab	Keyboard support
6	ARIA combobox attributes + ids via `useId` + live region	Screen-reader support
7	`onSearch` → `ProductResults` with loading/error/Retry, default products	Results page
8	`useRecentSearches` + `addRecent` + show on empty focus	History
9	Remove recent (× and Delete key)	Manage history

#	Step	Working result
10	`splitMatch` highlighting + messages (searching / none / error)	Polished dropdown
11	Scroll the active option into view; hover = highlight	Long lists usable
12	Node self-check for `addRecent` / `parseRecent` / `splitMatch`	Logic proven

9. Cover all states and edge cases

State / edge case	Handling
Idle (closed)	Input only; default products below
Focus, empty, history	Recent searches dropdown
Focus, empty, no history	Closed (nothing to show)
1 character	Closed; no request
Waiting for the debounce / loading	"Searching..." (or the previous list while loading)
Success	Up to 8 options, highlighted match, live count
Exactly one match	Shown (no `> 1` bug)
No matches	`No suggestions for "x"`
Suggestion error	Message in the dropdown; next keystroke retries
Results loading / error / empty	Faded old cards / Retry / "No products found."
Highlight past the end (list shrank)	Treated as no highlight (`active` derived)
Fast typing	One debounced request; stale ones aborted
Repeated term	Cache hit
Special characters (`c++` , `&` , `%`)	`encodeURIComponent` + regex-free highlight
Duplicate recent with different case	Deduped; newest spelling kept
Corrupted / blocked localStorage	Empty history; app keeps working
Click outside / Tab	Closes
Two comboboxes on a page	`useId` keeps ids unique

10. Final structure

```
_11_autocomplete_search_bar/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css                  ← global reset
    ├── App.jsx                    ← submitted query + recent searches; wiring
    ├── App.css
```

```
    ├── api/
    │   └── products.js              ← fetchSuggestions, fetchProducts
    ├── utils/
    │   └── search.js                ← addRecent, parseRecent, splitMatch (pure)
    ├── hooks/
    │   ├── useDebounce.js
    │   ├── useSuggestions.js        ← abort + cache + status
    │   └── useRecentSearches.js     ← localStorage-backed history
    └── components/
        ├── Autocomplete.jsx         ← ARIA combobox: input + listbox + keyboard
        └── ProductResults.jsx       ← cards for the submitted query
```

11. Full working code

src/api/products.js

```
const BASE = "https://dummyjson.com/products/search";

async function getProducts(url, signal) {
  const response = await fetch(url, { signal });
  if (!response.ok) throw new Error(`Request failed (HTTP ${response.status}).`);
  const data = await response.json();
  if (!Array.isArray(data.products)) throw new Error("Unexpected response from the server.");
  return data.products;
}

// Small payload for the dropdown: GET /products/search?q=ph&limit=8&select=title,category
export const fetchSuggestions = (query, { signal } = {}) =>
  getProducts(`${BASE}?q=${encodeURIComponent(query)}&limit=8&select=title,category`, signal);

// Full cards for the results page. Empty query = default products.
export const fetchProducts = (query, { signal } = {}) =>
  getProducts(`${BASE}?
q=${encodeURIComponent(query)}&limit=12&select=title,description,price,thumbnail`, signal);
```

src/utils/search.js

```
export const MAX_RECENT = 5;

// Newest first, no duplicates (case-insensitive), at most MAX_RECENT.
export function addRecent(list, term) {
  const value = term.trim();
  if (!value) return list;
  return [value, ...list.filter((t) => t.toLowerCase() !== value.toLowerCase())].slice(0, MAX_RECENT);
}
```

```
// Untrusted localStorage value -> clean string list.
export function parseRecent(raw) {
  try {
    const value = JSON.parse(raw);
    return Array.isArray(value) ? value.filter((t) => typeof t === "string" && t.trim()).slice(0,
MAX_RECENT) : [];
  } catch {
    return [];
  }
}

// Split text around the first case-insensitive match, for <mark> highlighting.
// No RegExp, so queries like "c++" or "(" are safe.
export function splitMatch(text, query) {
  const q = query.trim();
  const index = q ? text.toLowerCase().indexOf(q.toLowerCase()) : -1;
  if (index === -1) return [{ text, match: false }];
  return [
    { text: text.slice(0, index), match: false },
    { text: text.slice(index, index + q.length), match: true },
    { text: text.slice(index + q.length), match: false },
  ].filter((part) => part.text);
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from "react";

export function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debounced;
}
```

src/hooks/useSuggestions.js

```
import { useEffect, useRef, useState } from "react";
import { fetchSuggestions } from "../api/products";

const IDLE = { status: "idle", suggestions: [], error: "" };
```

```
// Suggestions for an already-debounced term: aborts stale requests, caches by term.
export function useSuggestions(term) {
  const [state, setState] = useState(IDLE); // status: idle | loading | success | error
  const cache = useRef(new Map());

  useEffect(() => {
    const key = term.toLowerCase();
    if (!key) {
      setState(IDLE);
      return;
    }
    if (cache.current.has(key)) {
      setState({ status: "success", suggestions: cache.current.get(key), error: "" });
      return;
    }

    const controller = new AbortController();
    setState((prev) => ({ ...prev, status: "loading", error: "" })); // keep showing the previous list
    fetchSuggestions(key, { signal: controller.signal })
      .then((suggestions) => {
        cache.current.set(key, suggestions);
        setState({ status: "success", suggestions, error: "" });
      })
      .catch((err) => {
        if (err.name === "AbortError") return;
        setState({ status: "error", suggestions: [], error: "Couldn't load suggestions." });
      });
    return () => controller.abort();
  }, [term]);

  return state;
}
```

src/hooks/useRecentSearches.js

```
import { useEffect, useState } from "react";
import { addRecent, parseRecent } from "../utils/search";

const KEY = "recentSearches";

function readRecent() {
  try {
    return parseRecent(localStorage.getItem(KEY));
  } catch {
    return []; // storage blocked
  }
}
```

```
export function useRecentSearches() {
  const [recent, setRecent] = useState(readRecent);

  useEffect(() => {
    try {
      localStorage.setItem(KEY, JSON.stringify(recent));
    } catch {
      // storage blocked or full: history only lasts for this visit
    }
  }, [recent]);

  return {
    recent,
    add: (term) => setRecent((prev) => addRecent(prev, term)),
    remove: (term) => setRecent((prev) => prev.filter((t) => t !== term)),
  };
}
```

src/components/Autocomplete.jsx

```
import { useEffect, useId, useState } from "react";
import { useDebounce } from "../hooks/useDebounce";
import { useSuggestions } from "../hooks/useSuggestions";
import { splitMatch } from "../utils/search";

const MIN_LENGTH = 2;
const DEBOUNCE_MS = 300;

const Highlight = ({ text, query }) =>
  splitMatch(text, query).map((part, i) => (part.match ? <mark key={i}>{part.text}</mark> : <span key=
{i}>{part.text}</span>));

// WAI-ARIA combobox: focus stays in the input; the "active" option is tracked with aria-
activedescendant.
const Autocomplete = ({ recent, onRemoveRecent, onSearch }) => {
  const [query, setQuery] = useState("");
  const [isOpen, setIsOpen] = useState(false);
  const [activeIndex, setActiveIndex] = useState(-1);
  const listboxId = useId();
  const optionId = (i) => `${listboxId}-option-${i}`;

  const typed = query.trim();
  const debounced = useDebounce(typed, DEBOUNCE_MS);
  const term = debounced.length >= MIN_LENGTH ? debounced : "";
  const { status, suggestions, error } = useSuggestions(term);
```

```
  // Derived: what the dropdown shows right now.
  const mode = typed === "" ? "recent" : typed.length >= MIN_LENGTH ? "suggest" : "none";
  const options =
    mode === "recent"
      ? recent.map((t) => ({ key: `recent-${t}`, label: t, recent: true }))
      : mode === "suggest"
        ? suggestions.map((p) => ({ key: `product-${p.id}`, label: p.title, meta: p.category }))
        : [];
  const active = activeIndex < options.length ? activeIndex : -1;
  const isPending = mode === "suggest" && (typed !== term || status === "loading");

  let message = "";
  if (mode === "suggest" && options.length === 0) {
    if (isPending) message = "Searching...";
    else if (status === "error") message = error;
    else if (status === "success") message = `No suggestions for "${typed}"`;
  }
  const showPopup = isOpen && (options.length > 0 || message !== "");

  // Keep the keyboard-highlighted option visible in a scrolling list.
  useEffect(() => {
    if (showPopup && active >= 0) document.getElementById(optionId(active))?.scrollIntoView({ block:
"nearest" });
  });

  const select = (label) => {
    setQuery(label);
    setIsOpen(false);
    setActiveIndex(-1);
    onSearch(label);
  };

  const onKeyDown = (e) => {
    switch (e.key) {
      case "ArrowDown":
        e.preventDefault();
        setIsOpen(true);
        if (options.length) setActiveIndex(active + 1 >= options.length ? 0 : active + 1);
        break;
      case "ArrowUp":
        e.preventDefault();
        setIsOpen(true);
        if (options.length) setActiveIndex(active <= 0 ? options.length - 1 : active - 1);
        break;
      case "Enter":
        e.preventDefault();
        if (showPopup && active >= 0) select(options[active].label);
```

```
      else if (typed) select(typed);
      break;
    case "Escape":
      if (isOpen) setIsOpen(false);
      else setQuery(""); // second Escape clears the input
      setActiveIndex(-1);
      break;
    case "Delete":
      if (showPopup && active >= 0 && options[active].recent) {
        e.preventDefault();
        onRemoveRecent(options[active].label);
      }
      break;
    case "Tab":
      setIsOpen(false);
      break;
    default:
  }
};

return (
  <div className="autocomplete">
    <label htmlFor={`${listboxId}-input`} className="sr-only">
      Search products
    </label>
    <div className="input-wrap">
      <input
        id={`${listboxId}-input`}
        type="text"
        role="combobox"
        aria-expanded={showPopup}
        aria-controls={listboxId}
        aria-autocomplete="list"
        aria-activedescendant={showPopup && active >= 0 ? optionId(active) : undefined}
        autoComplete="off"
        placeholder="Search for products..."
        value={query}
        onChange={(e) => {
          setQuery(e.target.value);
          setIsOpen(true);
          setActiveIndex(-1);
        }}
        onFocus={() => setIsOpen(true)}
        onBlur={() => setIsOpen(false)}
        onKeyDown={onKeyDown}
      />
      {query && (
```

```
          <button type="button" className="clear" aria-label="Clear search" onClick={() =>
setQuery("")}>
            ×
          </button>
        )}
      </div>

      {showPopup && (
        <div className="popup">
          {mode === "recent" && <p className="popup-title">Recent searches</p>}
          {message && <p className="popup-message">{message}</p>}
          {options.length > 0 && (
            <ul id={listboxId} role="listbox" aria-label={mode === "recent" ? "Recent searches" :
"Suggestions"}>
              {options.map((option, i) => (
                <li
                  key={option.key}
                  id={optionId(i)}
                  role="option"
                  aria-selected={i === active}
                  className={i === active ? "option active" : "option"}
                  onMouseDown={(e) => e.preventDefault()} // keep focus in the input so blur doesn't
close first
                  onMouseEnter={() => setActiveIndex(i)}
                  onClick={() => select(option.label)}
                >
                  <span className="icon" aria-hidden="true">
                    {option.recent ? "🕘" : "🔍"}
                  </span>
                  <span className="label">{option.recent ? option.label : <Highlight text={option.label}
query={typed} />}</span>
                  {option.meta && <span className="meta">{option.meta}</span>}
                  {option.recent && (
                    <span
                      className="remove"
                      aria-hidden="true"
                      title="Remove (or press Delete)"
                      onClick={(e) => {
                        e.stopPropagation();
                        onRemoveRecent(option.label);
                      }}
                    >
                      ×
                    </span>
                  )}
                </li>
              ))}
```

```
            </ul>
          )}
        </div>
      )}

      <p className="sr-only" aria-live="polite">
        {showPopup && options.length > 0 ? `${options.length} ${mode === "recent" ? "recent searches" :
"suggestions"} available` : message}
      </p>
    </div>
  );
};

export default Autocomplete;
```

src/components/ProductResults.jsx

```
import { useEffect, useState } from "react";
import { fetchProducts } from "../api/products";

// Results for the submitted query. "" shows default products on first load.
const ProductResults = ({ query }) => {
  const [state, setState] = useState({ status: "loading", products: [], error: "" });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setState((prev) => ({ ...prev, status: "loading", error: "" }));
    fetchProducts(query, { signal: controller.signal })
      .then((products) => setState({ status: "success", products, error: "" }))
      .catch((err) => {
        if (err.name !== "AbortError") setState({ status: "error", products: [], error: err.message });
      });
    return () => controller.abort();
  }, [query, attempt]);

  const { status, products, error } = state;

  return (
    <section className="results" aria-busy={status === "loading"}>
      <h2>{query ? `Results for "${query}"` : "Popular products"}</h2>
      {status === "loading" && products.length === 0 && <p className="note">Loading products...</p>}
      {status === "error" && (
        <div role="alert" className="note error">
          {error}{" "}
          <button type="button" onClick={() => setAttempt((a) => a + 1)}>
            Retry
```

```
          </button>
        </div>
      )}
      {status === "success" && products.length === 0 && <p className="note">No products found.</p>}
      <ul className={`cards ${status === "loading" ? "stale" : ""}`}>
        {products.map((p) => (
          <li key={p.id} className="card">
            <img src={p.thumbnail} alt="" width="64" height="64" loading="lazy" />
            <div>
              <h3>{p.title}</h3>
              <p className="description">{p.description}</p>
              <p className="price">${p.price}</p>
            </div>
          </li>
        ))}
      </ul>
    </section>
  );
};

export default ProductResults;
```

src/App.jsx

```
import { useState } from "react";
import "./App.css";
import Autocomplete from "./components/Autocomplete";
import ProductResults from "./components/ProductResults";
import { useRecentSearches } from "./hooks/useRecentSearches";

function App() {
  const { recent, add, remove } = useRecentSearches();
  const [submitted, setSubmitted] = useState(""); // the search the results page shows

  const handleSearch = (term) => {
    setSubmitted(term);
    add(term);
  };

  return (
    <main className="container">
      <h1>Product Search</h1>
      <Autocomplete recent={recent} onRemoveRecent={remove} onSearch={handleSearch} />
      <ProductResults query={submitted} />
    </main>
  );
}
```

```
export default App;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; color: #111827; }
.container { max-width: 720px; margin: 32px auto; padding: 0 16px; }
h1 { text-align: center; font-size: 26px; margin-bottom: 16px; }

.autocomplete { position: relative; }
.input-wrap { position: relative; }
.input-wrap input { width: 100%; padding: 12px 40px 12px 14px; font-size: 16px; border: 1px solid
#d1d5db; border-radius: 10px; background: white; }
.input-wrap input:focus { outline: 2px solid #4f46e5; border-color: transparent; }
.clear { position: absolute; right: 8px; top: 50%; transform: translateY(-50%); border: none;
background: none; font-size: 22px; color: #6b7280; cursor: pointer; }

.popup { position: absolute; z-index: 10; top: calc(100% + 4px); left: 0; right: 0; background: white;
border: 1px solid #e5e7eb; border-radius: 10px; box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1); overflow:
hidden; }
.popup ul { list-style: none; max-height: 320px; overflow-y: auto; }
.popup-title { padding: 8px 14px 4px; font-size: 12px; font-weight: 600; color: #6b7280; text-transform:
uppercase; }
.popup-message { padding: 12px 14px; color: #6b7280; }
.option { display: flex; align-items: center; gap: 10px; padding: 10px 14px; cursor: pointer; }
.option.active { background: #eef2ff; }
.option .label { flex: 1; }
.option mark { background: none; font-weight: 700; color: #4338ca; }
.option .meta { font-size: 12px; color: #9ca3af; text-transform: capitalize; }
.option .remove { color: #9ca3af; font-size: 18px; padding: 0 4px; }
.option .remove:hover { color: #dc2626; }

.results { margin-top: 24px; }
.results h2 { font-size: 18px; margin-bottom: 10px; }
.note { color: #6b7280; padding: 12px 0; }
.note.error { color: #dc2626; }
.note button { padding: 4px 10px; border: none; border-radius: 6px; background: #4f46e5; color: white;
cursor: pointer; }
.cards { list-style: none; display: grid; gap: 10px; transition: opacity 0.2s; }
.cards.stale { opacity: 0.5; }
.card { display: flex; gap: 12px; background: white; border: 1px solid #e5e7eb; border-radius: 10px;
padding: 12px; }
.card img { border-radius: 8px; object-fit: cover; background: #f3f4f6; flex-shrink: 0; }
.card h3 { font-size: 16px; }
.description { font-size: 14px; color: #4b5563; margin: 4px 0; display: -webkit-box; -webkit-line-clamp:
2; -webkit-box-orient: vertical; overflow: hidden; }
.price { font-weight: 700; color: #15803d; }
```

```
.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; }
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: min length, debounce, stale/out-of-order responses, caching, HTTP and shape errors, single-match dropdown, no matches, blur-vs-click, keyboard navigation with wrap, Escape twice, Delete to remove history, case-insensitive dedupe, corrupted storage, special characters, scroll into view, unique ids, live announcements.

Trade-offs to mention to the interviewer

- *Focus in the input (`aria-activedescendant`) vs moving focus to options:* typing keeps working and it matches the ARIA pattern; the cost is that you manage the highlight yourself.
- *Server suggestions vs a local list:* the server scales to any catalogue and ranks better, but adds latency (debounce + cache help). For small fixed lists, filter on the client instantly.
- *Prefix match vs "contains":* prefix feels predictable and is cheaper on the server; "contains" finds more. The API decides here; the highlight shows why an item matched.
- *Enter = search text vs Enter = first suggestion:* searching the text respects what the user typed; auto-selecting the first item is faster but surprising when it's wrong.
- *Remove button inside an option:* options shouldn't contain interactive children (ARIA), so × is mouse-only (`aria-hidden`) and keyboard users press Delete. A separate "Manage history" list is the strict alternative.

- *Selecting fills the input, which triggers one extra suggestion request* for the full title; it's cheap (and often cached), and it's simpler than skipping it with a flag.

What to add with more time

- Grouped results (Recent / Products / Categories) with `role="group"` headings.
- Inline completion (`aria-autocomplete="both"`): the rest of the top suggestion selected in the input.
- Popular/trending searches when there's no history; sync history to the account.
- Cache with TTL / LRU, or React Query; prefetch on hover.
- Tests: RTL + `user-event` for ↑/↓/Enter/Escape, `aria-activedescendant`, blur-vs-click, debounce with fake timers, mocked `fetch` for abort and errors.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Autocomplete vs search-as-you-type results: what's the difference?

Autocomplete suggests *queries or items* in a small dropdown while you type; you then pick one or press Enter. Search-as-you-type updates the full *results page* live. Here we do both: suggestions in the dropdown, results on selection.

Q2. Client-side or server-side suggestions?

Small, static lists (countries, tags): load once and filter on the client, instantly. Large or changing data (products, users): ask the server, which can rank, typo-correct and personalise. Then debounce, abort, cache and keep the payload small.

Q3. What should Enter do when nothing is highlighted?

Search exactly what the user typed (the safe default). Some products pick the first suggestion, but that can surprise users when it's not what they meant. Ask the interviewer.

State design

Q4. Why is the highlighted option an index in state and not DOM focus?

With `aria-activedescendant`, DOM focus stays in the input, so typing, the caret and selection keep working, while screen readers announce the highlighted option as if it were focused. Moving focus to options breaks typing and complicates returning focus.

Q5. Why derive `options` and `showPopup` instead of storing them?

They depend on `query`, `recent`, `suggestions` and `isOpen`. Storing them means syncing in effects, which lags one render and can show stale suggestions. Deriving guarantees the dropdown always matches the current input.

Q6. The list shrinks while an item is highlighted. What happens?

`active = activeIndex < options.length ? activeIndex : -1`, so an out-of-range index is treated as "no highlight". We also reset `activeIndex` to −1 whenever the user types.

Hard parts

Q7. Why does clicking a suggestion sometimes do nothing, and how do you fix it?

Event order: `mousedown` on the option → the input blurs → our `onBlur` closes the dropdown → the option is removed before `click`. Fix: `onMouseDown={e => e.preventDefault()}` on the options (the input keeps focus), then handle `onClick`. Alternatives: close on blur with a `relatedTarget` check, or a short delay (fragile).

Q8. Which ARIA attributes does a combobox need?

Input: `role="combobox"`, `aria-expanded`, `aria-controls` (listbox id), `aria-autocomplete="list"`, `aria-activedescendant` (highlighted option id). Popup: `role="listbox"` with `role="option"` children and `aria-selected` on the highlighted one. Plus a label for the input and a polite live region for counts or "no results".

Q9. How do you avoid showing results for an old query?

Debounce reduces requests, but responses can still arrive out of order. Abort the previous request when the term changes (`AbortController` in the effect cleanup) and ignore `AbortError`; or tag requests and ignore any response whose term isn't current.

Q10. How do you highlight the matching text safely?

Split the string with `indexOf` on lowercased text and wrap the match in `<mark>` as React elements. Avoid `new RegExp(userInput)` (it throws on `(` or `+` unless escaped) and avoid `dangerouslySetInnerHTML` (XSS from product titles or user input).

Q11. How would you highlight all matches, or fuzzy matches?

For all occurrences, loop with `indexOf` from the last index, or use a RegExp with the query escaped (`q.replace(/[.*+?^${}()|[\]\\]/g, "\\$&")`) and the `gi` flags. For fuzzy matching (typos, out-of-order letters) use a library like Fuse.js that returns match indices, or let the search server return highlight ranges.

Implementation steps

Q12. How do you store recent searches correctly?

Newest first, trimmed, deduped case-insensitively, capped (5), validated on read (array of non-empty strings), and wrapped in try/catch (blocked or corrupted storage). Don't store sensitive searches without consent; offer a way to remove items or clear all.

Q13. How would you test this component?

RTL + `user-event` with a mocked `fetch` and fake timers: type "pho" → advance 300 ms → expect 8 options and "8 suggestions available"; `{ArrowDown}{ArrowDown}` → expect `aria-activedescendant` to point to option 2; `{Enter}` → `onSearch` called with its label; click an option → no blur problem; `{Escape}` twice → closed then cleared; Delete removes a recent item.

Q14. How do you keep the highlighted option visible in a long, scrolling list?

After the highlight changes, call `element.scrollIntoView({ block: "nearest" })` on the active option (only scrolls if it's out of view). Don't scroll on mouse hover in a way that fights the user's pointer.

Performance and scale

Q15. Millions of products: what changes on the backend?

A dedicated search index (Elasticsearch / OpenSearch / Algolia) with prefix/edge n-gram analysis, ranking by popularity, typo tolerance, and a tiny response (id + title). Cache popular prefixes (CDN/Redis); the front end stays the same.

Q16. How do you make suggestions feel instant?

Cache by term, prefetch the next likely prefix, show cached results immediately while revalidating, keep the payload tiny, use HTTP/2 keep-alive, and show the previous list (faded) instead of a blank box while loading.

Q17. The dropdown renders 500 suggestions. Problem?

Yes: a long DOM list slows typing, and nobody scans 500 options. Cap the count (8-10) and add "See all results" (submit). If you really must show many, virtualise the list, keeping `aria-setsize` / `aria-posinset` correct.

Trade-offs and "why not X?"

Q18. Why not the native `<datalist>`?

`<datalist>` gives free suggestions, but you can't style it, can't highlight matches, can't show categories/icons or remove items, and behaviour differs across browsers and on mobile. Fine for simple forms, not for a product search.

Q19. Why not a library (Downshift, React Aria `useComboBox`, Headless UI Combobox)?

In production, yes: they implement the full ARIA pattern, edge cases (IME composition, mobile, screen readers) and are tested. In an interview, building it shows you understand focus management, blur-vs-click, `aria-activedescendant` and async correctness.

Q20. How would you handle IME input (Chinese/Japanese)?

While composing, `onChange` fires with partial characters. Check `e.nativeEvent.isComposing` (or `compositionstart` / `compositionend`) and don't fetch, or don't treat Enter as select, until composition ends. Otherwise Enter that confirms a character also submits the search.

Q21. Should recent searches be saved per device or per account?

Per device (localStorage) is private and simple but doesn't follow the user. Per account syncs across devices but needs privacy controls (clear history, incognito). Many apps do both: show local history immediately, merge with the server's history after login.